Практичне заняття № 3

Тема: Побудова ієрархії класів. Успадкування та перевантаження методів у Java

Мета заняття: навчитися будувати прості ієрархії класів у Java; створювати базові та похідні класи; використовувати *extends* і *super*; розуміти порядок виклику конструкторів; успадковувати поля та методи; реалізовувати перевантаження методів (*method overloading*) з різною кількістю та типами параметрів; закріпити роботу з об'єктами, масивами об'єктів, *static*, *enum*, умовами та циклами.

Короткі теоретичні положення

Успадкування. Успадкування дає змогу створити новий клас на основі вже існуючого. Базовий клас містить спільні властивості та поведінку, а похідний клас доповнює їх власними.

Ключове слово extends. Для створення похідного класу використовується *extends: class Student extends Person.* Student успадковує доступні члени Person.

Ключове слово super. *super* використовується для звернення до членів базового класу. Виклик *super(...)* у конструкторі похідного класу передає дані конструктору базового класу.

Перевантаження методів. Перевантаження (*overloading*) – це створення в одному класі кількох методів з однаковою назвою, але різними списками параметрів. Методи можуть відрізнятися кількістю, типами або порядком параметрів.

Важливо. Лише зміна типу результату не створює перевантаження. Сигнатура виклику має відрізнятися параметрами.

Overloading і overriding. Перевантаження – кілька однойменних методів із різними параметрами. Перевизначення (*overriding*) – нова реалізація успадкованого методу з тією самою сигнатурою.

Приклад 1. Базовий і похідний класи

```
class Person {
    String name;
    int age;
```

```
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void showInfo() {
        System.out.println(name + ", " + age + " років");
    }
}

class Student extends Person {
    String group;

    Student(String name, int age, String group) {
        super(name, age);
        this.group = group;
    }
}
```

Тут Student успадковує поля name, age і метод showInfo() класу Person. Поле group належить уже самому Student.

Приклад 2. Перевантаження методів

```
class Calculator {

    static int sum(int a, int b) {
        return a + b;
    }

    static int sum(int a, int b, int c) {
        return a + b + c;
    }

    static double sum(double a, double b) {
        return a + b;
    }
}
```

Назва методу однакова – sum, але Java вибирає потрібну версію за переданими аргументами.

Приклад 3. Що НЕ є перевантаженням

```
// Так НЕ можна:
int calculate(int x) {
```

```
    return x * 2;
}

// Помилка: відрізняється лише тип результату
double calculate(int x) {
    return x * 2.0;
}
```

Теоретичні відомості

1. Відношення «є різновидом». Успадкування доцільне тоді, коли об’єкт похідного класу справді є різновидом об’єкта базового класу. Наприклад, Student є Person, Car є Transport, Book є Publication. Якщо такого логічного зв’язку немає, успадкування використовувати не слід.

2. Що отримує похідний клас. Похідний клас може використовувати доступні поля та методи базового класу і водночас додавати власні. Це дає змогу не дублювати спільний код у кількох класах.

3. Модифікатори доступу в ієрархії. public-члени доступні з інших класів; private-члени безпосередньо доступні лише всередині класу, де вони оголошені; protected-члени доступні також у похідних класах. Для початкових прикладів достатньо розуміти, що private захищає внутрішні дані класу, а protected може бути зручним для успадкування.

4. Конструктори та успадкування. Конструктори не успадковуються. Кожен клас має власний конструктор. Під час створення об’єкта похідного класу спочатку виконується конструктор базового класу, а потім — похідного.

5. super(...). Виклик *super(...)* повинен бути першим оператором конструктора похідного класу. Він використовується для виклику відповідного конструктора базового класу.

6. Перевантаження методів. Перевантажені методи мають однакову назву, але різні списки параметрів. Можна змінювати кількість параметрів, їх типи або порядок типів. Це дозволяє виконувати близькі за змістом дії одним зрозумілим ім’ям методу.

7. Як Java обирає потрібний метод. Під час виклику аналізується кількість і типи переданих аргументів та підбирає відповідну сигнатуру. Наприклад, calculate(5) і calculate(5, 10) викликають різні перевантажені версії.

8. Тип результату не визначає перевантаження. Два методи не можна перевантажити лише зміною int на double у типі результату, якщо назва та параметри однакові. Для компілятора такі методи мають однакову сигнатуру виклику.

Схема простої ієрархії

Person

↓

Student

У цій схемі Person – базовий клас, а Student – похідний. Student успадковує спільні характеристики Person і додає власні.

Приклад 4. Як працює super

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
        System.out.println("Конструктор Person");
    }
}

class Student extends Person {
    String group;

    Student(String name, String group) {
        super(name);
        this.group = group;
        System.out.println("Конструктор Student");
    }
}
```

Якщо створити new Student("Anna", "AC-251"), спочатку виконається конструктор Person, а потім Student.

Приклад 5. Успадкований метод

```
class Animal {
    void sleep() {
        System.out.println("Тварина спить");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Мяу!");
    }
}

public class Main {
    public static void main(String[] args) {
        Cat cat = new Cat();
        cat.sleep(); // успадкований метод
        cat.meow();  // власний метод Cat
    }
}
```

Приклад 6. Перевантаження за кількістю параметрів

```
class Printer {
    void print(String text) {
        System.out.println(text);
    }

    void print(String text, int count) {
        for (int i = 0; i < count; i++) {
            System.out.println(text);
        }
    }
}
```

У першому випадку print() отримує один параметр, у другому – два. Це коректне перевантаження.

Приклад 7. Перевантаження за типами параметрів

```
class Converter {
    double convert(int value) {
        return value * 1.0;
    }

    double convert(double value) {
```

```
        return value;
    }

    String convert(String value) {
        return value.toUpperCase();
    }
}
```

Кількість параметрів однакова, але їх типи різні: int, double і String.

Приклад 8. Перевантаження за порядком параметрів

```
class Message {
    void show(String text, int number) {
        System.out.println(text + " " + number);
    }

    void show(int number, String text) {
        System.out.println(number + " " + text);
    }
}
```

Типи параметрів ті самі, але їх порядок різний, тому сигнатури методів відрізняються.

Приклад 9. Невелика ієрархія Employee → Programmer

```
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void showInfo() {
        System.out.println(name + ": " + salary + " грн");
    }
}

class Programmer extends Employee {
    String language;

    Programmer(String name, double salary, String language)
{
        super(name, salary);
```

```
        this.language = language;
    }

    double calculateSalary() {
        return salary;
    }

    double calculateSalary(double bonus) {
        return salary + bonus;
    }

    double calculateSalary(double bonus, double premium) {
        return salary + bonus + premium;
    }
}
```

Цей приклад одночасно демонструє успадкування, super і три перевантажені версії calculateSalary().

Типові помилки

- Забути extends – класи не утворюватимуть ієрархію.
- Не передати потрібні дані через super(...) у конструктор базового класу.
- Створити два методи з однаковими параметрами, змінивши лише тип результату – це помилка компіляції.
- Плутати перевантаження з перевизначенням: у цій роботі перевантажені методи повинні відрізнятися параметрами.
- Дублювати в похідному класі поля, які вже є в базовому, замість використання успадкування.

Міні-вправа перед індивідуальним завданням

Визначте, які пари класів утворюють логічну ієрархію успадкування:

- Person → Student
- Transport → Car
- Book → Author
- Device → Smartphone
- Animal → Dog

Поясніть, чому зв’язок Book → Author не є відношенням «є різновидом».

Повний приклад: система транспорту

Створимо базовий клас Transport і похідний клас Car. Для розрахунку вартості поїздки використаємо перевантажений метод calculateCost().

```
enum FuelType {
    PETROL, DIESEL, ELECTRIC
}

class Transport {
    String brand;
    int year;

    Transport(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    void showInfo() {
        System.out.println("Марка: " + brand + ", рік: " +
year);
    }
}

class Car extends Transport {
    FuelType fuelType;

    Car(String brand, int year, FuelType fuelType) {
        super(brand, year);
        this.fuelType = fuelType;
    }

    double calculateCost(double km) {
        return km * 8;
    }

    double calculateCost(double km, double pricePerKm) {
        return km * pricePerKm;
    }

    double calculateCost(double km, double pricePerKm,
double discount) {
```

```
        return km * pricePerKm * (1 - discount);
    }

    void showCarInfo() {
        showInfo();
        System.out.println("Тип пального: " + fuelType);
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car("Toyota", 2022, FuelType.PETROL);

        car.showCarInfo();

        System.out.println("50 км: " +
car.calculateCost(50));
        System.out.println("50 км за тарифом 10 грн: "
                + car.calculateCost(50, 10));
        System.out.println("Зі знижкою 10%: "
                + car.calculateCost(50, 10, 0.10));
    }
}
```

На що варто звернути увагу в цьому прикладі

- Transport – базовий клас, Car – похідний клас;
- Car отримує brand, year і showInfo() через успадкування;
- super(brand, year) викликає конструктор базового класу;
- fuelType – власне поле класу Car і має тип enum FuelType;
- calculateCost() перевантажено тричі;
- Java визначає потрібну версію calculateCost() за кількістю переданих аргументів.

Хід виконання роботи

За індивідуальним варіантом створити невелику програму з ієрархією класів.

Програма повинна:

- містити базовий клас і щонайменше один похідний клас;
- використовувати extends;

- мати щонайменше два поля у базовому класі та одне додаткове поле у похідному;
- мати конструктори базового та похідного класів;
- використовувати *super(...)* у конструкторі похідного класу;
- містити звичайний метод для виведення інформації про об’єкт;
- містити один метод, перевантажений щонайменше у трьох варіантах;
- використовувати щонайменше одне enum там, де це природно для предметної області;
- створити не менше трьох об’єктів похідного класу;
- помістити об’єкти в масив і опрацювати його циклом;
- використати умову if або switch;
- продемонструвати виклики різних версій перевантаженого методу.

Варіанти індивідуальних завдань

Варіант	Мініпроєкт	Ієрархія	enum	Основне завдання
B.1	**Люди в університеті**	Person → Student	enum StudyForm { BUDGET, CONTRACT }	Перевантажити calculateScholarship(): за середнім балом; за балом і базовою стипендією; за балом, базовою стипендією та коефіцієнтом. Вивести студентів заданої форми навчання.
B.2	**Працівники ІТ-компанії**	Employee → Programmer	enum Level { JUNIOR, MIDDLE, SENIOR }	Перевантажити calculateSalary(): базова зарплата; зарплата + бонус; зарплата + бонус + премія. Вивести програмістів певного рівня.
B.3	**Транспорт**	Transport → Car	enum FuelType { PETROL, DIESEL, ELECTRIC }	Перевантажити calculateCost() для різних способів розрахунку вартості поїздки. Вивести автомобілі заданого типу пального.
B.4	**Бібліотека**	Publication → Book	enum Genre { PROGRAMMING, SCIENCE, FICTION }	Перевантажити calculatePrice(): звичайна ціна; зі знижкою; зі знижкою та доставкою.

				Вивести книги заданого жанру.
B.5	**Інтернет-магазин**	Product → ElectronicProduct	enum Category { PHONE, LAPTOP, ACCESSORY }	Перевантажити finalPrice(): ціна; ціна зі знижкою; ціна зі знижкою та доставкою. Знайти товари заданої категорії.
B.6	**Тварини**	Animal → Dog	enum Size { SMALL, MEDIUM, LARGE }	Перевантажити feed(): порція за вагою; за вагою й віком; за вагою, віком і активністю. Вивести собак заданого розміру.
B.7	**Навчальні курси**	Course → OnlineCourse	enum Level { BEGINNER, INTERMEDIATE, ADVANCED }	Перевантажити calculateFee(): базова вартість; зі знижкою; зі знижкою та сертифікатом. Вивести курси заданого рівня.
B.8	**Банківські рахунки**	Account → SavingsAccount	enum Currency { UAH, USD, EUR }	Перевантажити calculateInterest(): за балансом; балансом і ставкою; балансом, ставкою та строком. Вивести рахунки заданої валюти.
B.9	**Кінотеатр**	Media → Movie	enum Genre { COMEDY, DRAMA, ACTION }	Перевантажити ticketPrice(): стандартний квиток; квиток із коефіцієнтом; квиток із коефіцієнтом і знижкою. Вивести фільми заданого жанру.
B.10	**Готель**	Accommodation → HotelRoom	enum RoomType { STANDARD, DELUXE, SUITE }	Перевантажити calculateStay(): за кількістю днів; за днями і ціною; за днями, ціною і знижкою. Вивести номери заданого типу.
B.11	**Ресторан**	MenuItem → Dish	enum DishType { STARTER, MAIN, DESSERT }	Перевантажити calculateOrder(): ціна; ціна × кількість; ціна × кількість + сервіс. Вивести страви заданого типу.
B.12	**Фітнес-клуб**	Person → ClubMember	enum Membership { BASIC, STANDARD, PREMIUM }	Перевантажити calculatePayment(): місяць; кілька місяців; кілька місяців зі знижкою. Вивести клієнтів заданого абонемента.

B.13	**Смарт-пристрої**	Device → Smartphone	enum OS { ANDROID, IOS, OTHER }	Перевантажити calculatePrice(): базова ціна; з пам’яттю; з пам’яттю та страховкою. Вивести смартфони заданої ОС.
B.14	**Доставка**	Delivery → CourierDelivery	enum Speed { STANDARD, EXPRESS, SAME_DAY }	Перевантажити calculateDelivery(): за відстанню; за відстанню й вагою; за відстанню, вагою та швидкістю. Вивести доставки заданого типу.
B.15	**Комп’ютерні ігри**	Software → Game	enum GameGenre { ACTION, STRATEGY, PUZZLE }	Перевантажити calculateRating(): одна оцінка; три оцінки; масив оцінок. Вивести ігри заданого жанру.
B.16	**Авіаперельоти**	Transport → Flight	enum FlightClass { ECONOMY, BUSINESS, FIRST }	Перевантажити ticketPrice(): базова ціна; ціна з багажем; ціна з багажем і класом. Вивести рейси заданого класу.
B.17	**Музика**	Media → Song	enum MusicGenre { POP, ROCK, CLASSICAL }	Перевантажити playInfo(): назва; назва й виконавець; назва, виконавець і тривалість. Вивести композиції заданого жанру.
B.18	**Автосалон**	Vehicle → ElectricCar	enum BatteryType { LFP, NMC, OTHER }	Перевантажити calculateRange(): за ємністю; за ємністю й витратою; з додатковим коефіцієнтом. Вивести авто заданого типу батареї.
B.19	**Система заявок**	Message → SupportTicket	enum Priority { LOW, MEDIUM, HIGH }	Перевантажити estimateTime(): за пріоритетом; за пріоритетом і складністю; з додатковим коефіцієнтом. Вивести заявки високого пріоритету.
B.20	**Геометричні фігури**	Shape → Rectangle	enum Unit { MM, CM, M }	Перевантажити area(): за стороною квадрата; за двома сторонами прямокутника; за двома сторонами та коефіцієнтом масштабу. Вивести результати для різних одиниць.

Завдання «Передбач результат»

```
class A {
    A() {
        System.out.println("A");
    }
}

class B extends A {
    B() {
        System.out.println("B");
    }
}

public class Main {
    public static void main(String[] args) {
        B object = new B();
    }
}
```

До запуску програми запишіть порядок виведення символів і поясніть, чому конструктори викликаються саме так.

Завдання «Знайди правильне перевантаження»

```
class Example {
    void print(int x) { }
    void print(int x, int y) { }
    void print(double x) { }

    // Чи можна додати?
    // int print(int x) { return x; }
}
```

Поясніть, які методи є перевантаженими і чому останній закоментований метод спричинить помилку.

До звіту необхідно включити

- тему та мету практичного заняття;
- умову індивідуального завдання;
- схему ієрархії класів у вигляді BaseClass → DerivedClass;
- лістинг усіх створених класів та enum;
- пояснення використання extends і super;

- перелік перевантажених версій методу та пояснення, чим відрізняються їхні параметри;
- скріншот результату виконання програми;
- відповідь на одне з додаткових завдань;
- висновки.

У висновках потрібно

- зазначити, який клас є базовим, а який похідним;
- описати, які поля та методи успадковуються;
- пояснити призначення super;
- пояснити принцип перевантаження методів;
- зазначити, за якими параметрами Java обирає потрібну версію методу;
- коротко проаналізувати результат роботи програми.

Контрольні питання

1. Що називається успадкуванням у Java?
2. Що таке базовий клас?
3. Що таке похідний клас?
4. Для чого використовується extends?
5. Що успадковує похідний клас від базового?
6. Для чого використовується super?
7. Що означає виклик super(...) у конструкторі?
8. У якому порядку викликаються конструктори базового та похідного класів?
9. Що таке перевантаження методів?
10. Які методи можна вважати перевантаженими?
11. Чи можуть перевантажені методи мати різну кількість параметрів?
12. Чи можуть вони мати різні типи параметрів?
13. Чи достатньо змінити лише тип результату методу для перевантаження?
14. Як Java визначає, яку перевантажену версію методу викликати?
15. Чим overloading відрізняється від overriding?

16. Чи успадковуються конструктори?
17. Чи може похідний клас мати власні поля?
18. Як створити об’єкт похідного класу?
19. Як створити масив об’єктів похідного класу?
20. Наведіть власний приклад ієрархії класів.